Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Automatic Differentiation

Motivation



- Calculating derivatives is the crucial step in all the training algorithms
- Modern deep learning frameworks automate this process by offering automatic differentiation
- Two crucial ideas are back-propagation method and computational graphs
- Computational graph tracks how each value depends on others and automatic differentiation works backwards through this graph applying the chain rule (back-propagation)

Back-propagation for MLP



- Consider our canonical training task

$$\min_w f(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(w, z_i) \tag{1}$$

for a simple MLP and a regression task

$$\ell(w, z_i) = \frac{1}{2}\|y_i - W_L\sigma(W_{L-1}\dots W_2\sigma(W_1 x_i))\|^2, \quad x_i \in \mathbb{R}^p, \quad y_i \in \mathbb{R}^m \tag{2}$$

- Thus, our learnable parameters are $w = (W_1, \dots, W_L) \in \mathbb{R}^d$
- And SGD update is $w_{t+1} = w_t - \eta g_t$ with g_t representing the gradient of the mini batch used in iteration t
- Our goal is to find partial derivatives w.r.t. each weight matrix using chain rule (treating each W_l as a $d_l \times d_{l-1}$ column vector such that $\partial_{W_L}\ell$ is also a $d_l \times d_{l-1}$ column vector).

Back-propagation: defining Variables



- Define pre-activation vectors v and post-activation vectors h

$$v_l = W_l h_{l-1}, \quad h_l = \sigma(v_l), \quad v_l, h_{l-1} \in \mathbb{R}^{d_l}, \quad l = 1, \ldots, L \tag{3}$$

where $h_0 = x \in \mathbb{R}^d$ and d_l denotes the number of neurons in layer l. Note v_L denotes model's prediction (output layer).

- Define the final error signal

$$e_L = \partial_{v_L} \ell = \partial_{v_L} \frac{1}{2} \|y - v_L\|^2 = v_L - y, \quad e_L \in \mathbb{R}^{d_L} \tag{4}$$

- Define activation derivative matrix

$$D_l = \text{diag}(\sigma'(v_l(1)), \ldots \sigma'(v_l(d_l))) \in \mathbb{R}^{d_l \times d_l} \tag{5}$$

Partial Derivatives



- We can represent each partial derivative using our defined variables. For instance,

$$\partial_{W_L}\ell = \partial_{W_L} v_L \times \partial_{v_L}\ell = \underbrace{I_{d_{L-1}} \otimes h_{L-1}}_{(d_L \cdot d_{L-1}) \times d_L \text{ matrix (Kronecker product)}} \times \underbrace{v_L - y}_{= e_L,\ d_L \times 1 \text{ vector}} \tag{6}$$

- Example:

$$a = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \in \mathbb{R}^2, \quad W = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix} \in \mathbb{R}^{3\times 2}.$$

$$I_3 \otimes a = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 2 \end{bmatrix} \in \mathbb{R}^{6\times 3}.$$

Partial Derivatives (Cont'd)

- Recall $v_l = W_l h_{l-1}, \quad h_l = \sigma(v_l), \quad v_l, h_{l-1} \in \mathbb{R}^{d_l}, \quad l = 1, \ldots, L$
- For layer $L-1$

$$\partial_{W_{L-1}} \ell = \underbrace{\partial_{W_{L-1}} v_{L-1}}_{h_{L-2}} \times \underbrace{\partial_{v_{L-1}} h_{L-1}}_{\text{derivative of activation } D_{L-1}} \times \underbrace{\partial_{h_{L-1}} v_L}_{W_L} \times \underbrace{\partial_{v_L} \ell}_{e_L} \tag{7}$$

where we may need operations like $I\otimes$ and $^\top$ to ensure dimensions match and the final answer is a $d_{L-1} \cdot d_{L-2}$ vector

- Consider defining

$$e_{L-1} := \partial_{v_{L-1}} \ell = \underbrace{\partial_{v_{L-1}} h_{L-1}}_{\text{derivative of activation } D_{L-1}} \times \underbrace{\partial_{h_{L-1}} v_L}_{W_L} \times \underbrace{\partial_{v_L} \ell}_{e_L} \tag{8}$$

by chain rule

Partial Derivatives (Cont'd)



- Thus

$$\partial_{W_{L-1}} \ell = \underbrace{\partial_{W_{L-1}} v_{L-1}}_{h_{L-2}} \times e_{L-1} \tag{9}$$

that is the partial derivative for each layer l is just the product of the activation/output of the previous layer h_{l-1} and the error of that layer e_l

- Gives us a natural procedure to leverage memory to reduce computational cost
- This procedure is called Back-propagation:
  - compute pre/post activations v_l, h_l in the forward pass and store them
  - compute and propagate the error signals in the backward pass

$$e_{l-1} = D_{l-1} \times W_l \times e_l, \quad l = 1, \ldots, L \tag{10}$$

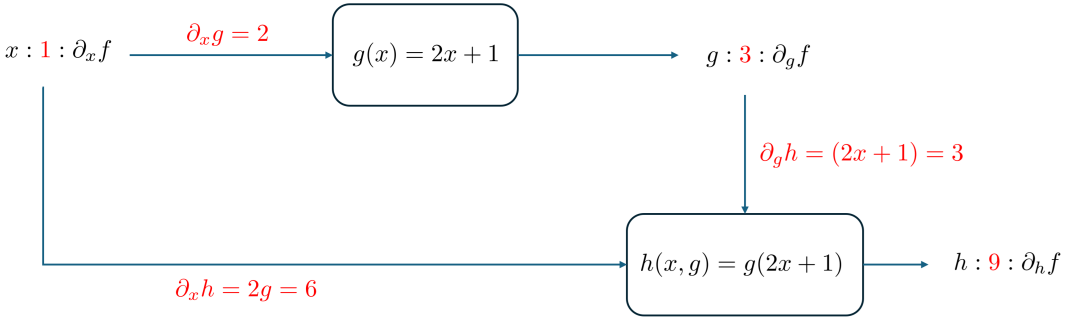
Computation Graph



- A computation graph is a directed acyclic graph (DAG) that represents the sequence of operations applied to variables in a computational framework, such as deep learning.
- Computational graph tracks how each value depends on others and automatic differentiation works backwards through this graph applying the chain rule (back-propagation)
- Example $f(x) = (2x + 1) \cdot (2x + 1)$
- We think of this as $h(x, g) = g(2x + 1)$ where $g(x) = 2x + 1$
- We aim to calculate the derivative of f at $x = 1$

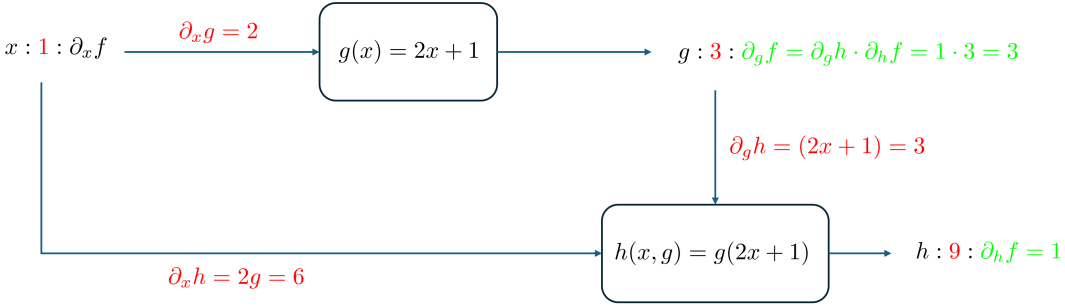
Computation Graph Example

- Forward pass
  - find the values of input variables (pre/post activations h_l, v_l)
  - label the edges with relevant partial derivatives (activation derivatives D_l)



Computation Graph Example (Cont'd)

- Backward pass
  - move backward to calculate the derivatives w.r.t. activations (error signals e_l's)
  - accumulate the derivatives and sum over all path



$$\partial_x f = \partial_x h \cdot \partial_h f + (\partial_h f \cdot \partial_g h) \cdot \partial_x g = 1 \cdot 6 + 2 \cdot (3 \cdot 1) = 6 + 6 = 12$$

Static Computation Graph



- The entire computation is defined and compiled before execution (a define-then-run approach), e.g. earlier versions of Tensorflow
- Advantages
  - More optimizable, as the entire computation graph is known in advance.
  - Allows for efficient graph-level optimizations, such as operation fusion
  - Can be deployed more easily for inference after compilation.
- Disadvantages
  - Less flexible: modifying the graph requires redefining and recompiling it
  - Debugging is harder since the execution is decoupled from definition.

Dynamic Computation Graph



- Built on-the-fly during execution. The structure of the graph can change at each iteration (a define-by-run approach), e.g. Tensorflow 2.0 and Pytorch
- Advantages
  - Highly flexible: enables dynamic control flows, such as loops and conditionals.
  - Easier to debug because the execution happens immediately.
  - More intuitive for model development, especially for recurrent networks and reinforcement learning.
- Disadvantages
  - Potentially less optimized since the entire graph is not known beforehand.
  - Can be slower compared to static graphs due to runtime overhead.